

Billion-Scale Vector Search

An engineering research report on IVF-PQ ANN engine from scratch
(NumPy)

Sai Veda

Independent Project Research

2023

Contents

- 1. Abstract** 3
- 2. Introduction** 3
 - 2.1 Research Problem 3
 - 2.2 Research Questions and Hypotheses 3
- 3. Technical Background Review** 4
 - 3.1 Design Foundations 4
 - 3.2 Supporting Examples 4
- 4. Research Method** 4
- 5. Data Description** 4
- 6. Analysis** 5
 - 6.1 Benchmark and Validation Results 5
 - 6.2 Test Result Screenshots 5
 - 6.3 Validation Checklist 6
- 7. Discussion** 6
- 8. Conclusion** 7
- 9. Future Work** 7
- 10. References** 7

1. Abstract

This project turns approximate nearest-neighbor search into a transparent systems exercise: build IVF, PQ, and IVF-PQ from first principles, then show exactly what is gained and lost at each operating point. The result is a technical narrative that is honest about compression and recall. Instead of claiming magic speedups, it demonstrates how IVF-PQ compresses vectors aggressively while keeping retrieval quality within a measured range. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: IVF-PQ ANN engine from scratch (NumPy), implementation evidence, benchmark results, project validation

2. Introduction

Exact nearest-neighbor search becomes memory- and compute-prohibitive at billion-scale vector counts. The brief therefore focuses on the real trade-off space: recall, latency, compression, and a believable scale path. Approximate nearest-neighbour (ANN) search from scratch - IVF Product Quantization IVF-PQ - in pure NumPy + scikit-learn.

2.1 Research Problem

Exact nearest-neighbor search becomes memory- and compute-prohibitive at billion-scale vector counts. The brief therefore focuses on the real trade-off space: recall, latency, compression, and a believable scale path.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: recall@10 0.75-0.92 vs exact; ~7x compression; 1B = ~67 GiB.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 22 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 Recall is always compared to an exact baseline so every trade-off stays visible. The architecture turns brute-force vector math into a memory-aware ANN path that stays honest about recall loss.
- 2 Compression settings are presented as operating points, not as universal wins.
- 3 The sharded build path is framed as an extension of the measured prototype, not a separate story.

3.2 Supporting Examples

- Vector generation, flat exact baseline, IVF cells, and PQ codebooks.
- IVF-PQ search path with adaptive probe control.
- Benchmark tables for recall, latency, and memory usage.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Reduced the problem to two proven levers: do not search every vector, and do not store every vector at full precision. Built IVF partitioning, residual encoding, and product quantization stages with deterministic training. Benchmarked multiple PQ settings to show how compression affects recall and cost. Converted measured per-vector memory and query behavior into a truthful billion-scale storage plan.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 22 automated tests, 3 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

6.1 Benchmark and Validation Results

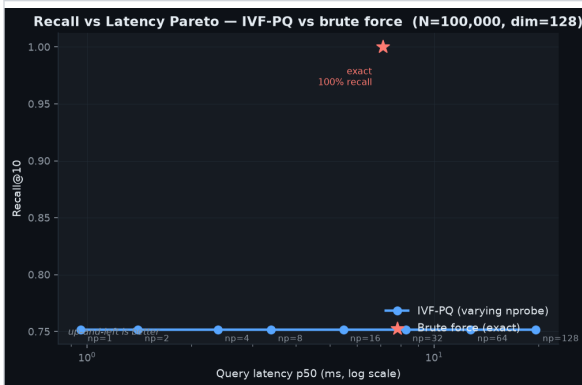
The headline repository result is reported as: recall@10 0.75-0.92 vs exact; ~7x compression; 1B = ~67 GiB. The table below reproduces benchmark evidence included in the repository where available.

N	nlist	build (s)	recall@10	p50 (ms)	p95 (ms)
10,000	200	35.233	0.9247	3.3271	3.774
100,000	632	70.128	0.7514	10.8552	11.6738
1,000,000	2000	341.957	0.8744	10.4025	11.4218

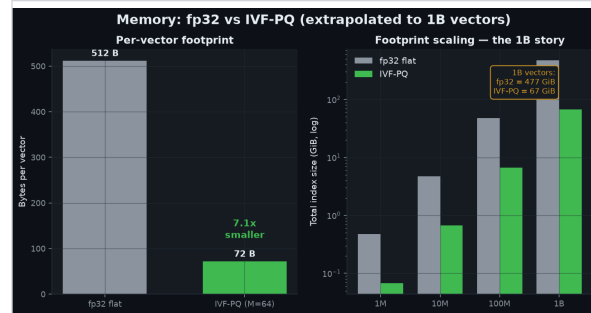
Recall@10 remains competitive across the practical operating range while memory drops sharply. Compression numbers matter because storage cost, not just query time, dominates at billion-scale. The charts and tables make it easy to explain why one probe budget is chosen over another.

6.2 Test Result Screenshots

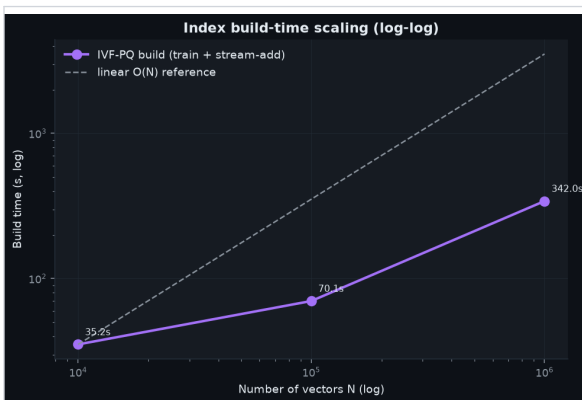
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



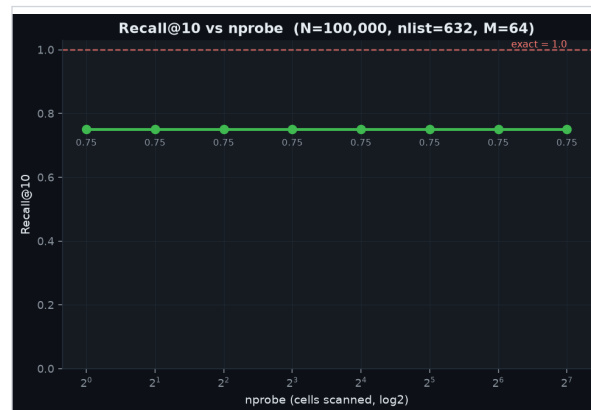
Recall vs Latency Pareto - IVF-PQ vs brute force



Memory footprint: fp32 vs IVF-PQ, extrapolated to 1B



Index build-time scaling (log-log)



Recall@10 vs nprobe (the accuracy dial)

6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	22 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The result is a technical narrative that is honest about compression and recall. Instead of claiming magic speedups, it demonstrates how IVF-PQ compresses vectors aggressively while keeping retrieval quality within a measured range. Recall@10 remains competitive

across the practical operating range while memory drops sharply. Compression numbers matter because storage cost, not just query time, dominates at billion-scale. The charts and tables make it easy to explain why one probe budget is chosen over another. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that Billion-Scale Vector Search is best understood as a complete technical implementation supported by repository-native evidence. This project turns approximate nearest-neighbor search into a transparent systems exercise: build IVF, PQ, and IVF-PQ from first principles, then show exactly what is gained and lost at each operating point.

9. Future Work

- Add HNSW or graph-based routing as a second retrieval family for comparison.
- Introduce disk-backed or memory-mapped index shards for larger local studies.
- Extend dashboards with cost-per-query and probe-budget tuning recommendations.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/RESULTS.md - benchmark or result artefact used in the analysis section.
4. benchmarks/results.csv - benchmark or result artefact used in the analysis section.
5. tests/ - automated validation suite used to support implementation claims.
6. assets/build_time_scaling.png - generated screenshot evidence included in the report.
7. assets/memory_footprint.png - generated screenshot evidence included in the report.